Lesson 9: On the Scoreboard

Godot Engine

VIDEO GAME CREATION

This week: track a real score, and show it - along with your lives - on screen, all the time, not just at the end.

Open your own [YourName]Game project from Lesson 8 - don't start a new one.

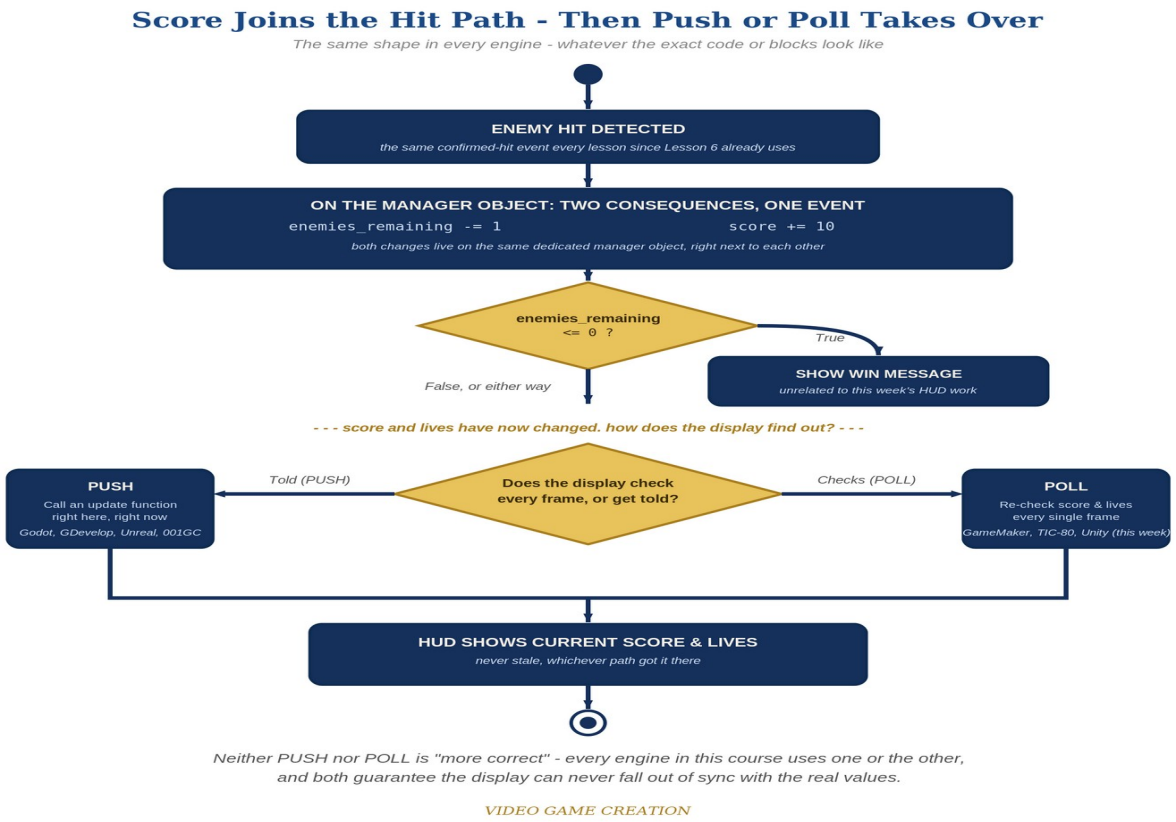
Tip: no new assets this week - this is a logic and UI lesson.

Setup

1. Open your [YourName]Game project from Lesson 8.
2. Add two Label nodes to your main scene: ScoreText and LivesText, positioned in a top corner where they won't overlap gameplay. Leave both visible (unlike WinText/LoseText, these stay on screen the whole game).

Level 1 - Minimal (MVP)

Destroying an enemy adds 10 points to your score. Score and lives are both visible on screen, all the time, not just at the start or end of the game.



Score joins the hit path, then Push or Poll takes over - the same shape in every engine, whatever the exact code or blocks look like.

3. In GameManager's script - the same node that already owns enemies_remaining, lives, on_enemy_destroyed(), and on_enemy_escaped() - add var score := 0.
4. Write a new function update_hud() on GameManager's script that sets both Labels' text from score and lives, reaching them via get_node("../ScoreText") and get_node("../LivesText") - the same relative-path pattern GameManager already uses to reach WinText and LoseText (code below).
5. Add a _ready() function to GameManager's script that calls update_hud() once, so the HUD shows correct starting values the instant the scene starts.
6. In on_enemy_destroyed(), add score += 10 and call update_hud(). In on_enemy_escaped(), call update_hud() too, right after decrementing lives.
7. Run the scene, shoot a few enemies, and let one escape - both labels should update immediately every time.

Highlighted lines are new this week; everything else is code you already have.

```
# GameManager's script (complete):
var enemies_remaining := 5
var lives := 3
var score := 0

func _ready():
    update_hud()

func update_hud():
    get_node("../ScoreText").text = "Score: " + str(score)
    get_node("../LivesText").text = "Lives: " + str(lives)
```

```
func on_enemy_destroyed():
    enemies_remaining -= 1
    score += 10
    update_hud()
    if enemies_remaining <= 0:
        get_node("../WinText").show()

func on_enemy_escaped():
    lives -= 1
    update_hud()
    if lives <= 0:
        get_node("../LoseText").show()
```

Level 2 - Complete (Alpha/Beta)

The score and lives readouts never show a stale or wrong number - they always match reality, immediately.

- Shoot 3 enemies and confirm ScoreText reads exactly "Score: 30".
- Let an enemy escape and confirm ScoreText does NOT change - only LivesText should move.
- Confirm both labels show correct values the instant the scene starts, not just after the first kill or escape (this is why update_hud() is called once in GameManager's own _ready()).

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Visual: use the Inspector's Theme Overrides on each Label to increase font size or change color for a more game-like HUD look.
- Visual: change points-per-kill to a randomized value (e.g. randi_range(5, 15)) for scoring variety.
- Structure: notice update_hud() is now the ONLY code that touches ScoreText or LivesText directly - every other function just changes the underlying numbers and calls update_hud() to reflect them. This separation (game logic vs. display) is a new idea this week, distinct from the hit-vs-escape separation from Lessons 6-8 - and it still lives entirely on GameManager, the same dedicated manager node that's owned all of this game's shared state since Lesson 6.

If Something's Wrong

- Labels show "Score: 0" forever: confirm update_hud() is actually being called inside on_enemy_destroyed(), not just defined and forgotten.
- Error referencing ScoreText or LivesText: confirm the Label node names in the Scene tree exactly match (case-sensitive) what's referenced in GameManager's get_node() calls, and that the Labels are children of the main scene (siblings of GameManager), not children of GameManager itself.
- HUD doesn't show correct values at the very start: confirm update_hud() is called once inside GameManager's own _ready() function, not only inside the two handler functions.

Want More?

Student Resource List > Godot section, or docs.godotengine.org